كلية هندسة الذكاء الاصطناعي
FACULTY OF ARTIFICIAL INTELLIGENCE ENGINEERING

Smart Family Budget Assistant

Powered by Large Language Models and Retrieval-Augmented Generation

Graduation Project 1

prepared to obtain the degree of B.Sc. in the Department of Artificial Intelligence Engineering and Data Science

Author

Ayham Al-Salem

Supervised by

Dr. Nasreen Suleiman

Eng. Zainab Baghdadi

دمشق: 1447-1448هـ

Damascus: 2025–2026

SUPERVISOR CERTIFICATION

I certify that the preparation of this project, entitled

"Smart Family Budget Assistant Powered by LLM and RAG,"

prepared by

Ayham Al-Salem

was carried out under my supervision at the Faculty of Artificial Intelligence Engineering, Syrian Private University, in partial fulfillment of the requirements for the degree of B.Sc. in the Department of Artificial Intelligence Engineering and Data Science.

Name: Dr. Nasreen Suleiman

Date: ……

Signature: ……

Name: Eng. Zainab Baghdadi

Date: ……

Signature: ……

الملخّص

في ظلّ الظروف الاقتصادية الراهنة في سوريا والمنطقة العربية، تواجه العائلات صعوبات حقيقية في تتبّع مصروفاتها اليومية واتخاذ قرارات مالية مدروسة. وعلى الرغم من توفّر العديد من تطبيقات إدارة الميزانية، فإنّ معظمها يعمل باللغة الإنجليزية ولا يدعم اللهجات المحلية، كما أنّ دقّة استخراج البيانات المالية من النصوص المكتوبة بلغة طبيعية لا تزال محدودة (38–78% وفقاً للدراسات السابقة).

وتقنية (LLM) يدمج بين نماذج اللغة الكبيرة (Mizan) "يقدّم هذا المشروع نظاماً ذكياً يُسمّى "وكيل الميزانية العائلي الذكي لإدارة المصروفات العائلية باللغة العربية واللهجة السورية. يعتمد النظام على نموذج (RAG) التوليد المعزّز بالاسترجاع لاستخراج المبلغ والفئة والتاريخ من جملٍ بسيطة كـ "دفعت 5000 بنزين"، ثمّ يقوم Groq عبر منصّة Llama 3.3 70B وقاعدة البيانات المتجهية BGE-M3 بتصنيفها تلقائياً ضمن 12 فئة رئيسية. كما يستخدم نموذج التضمين ChromaDB لاسترجاع البيانات السابقة وتوليد إجابات ذكية على استفسارات المستخدم.

أظهرت نتائج التقييم على مجموعة اختبار تتألّف من 180 جملة باللهجة السورية أداءً متميّزاً: بلغت دقّة استخراج المبلغ الكلّي 96.8%، متفوّقاً على الدراسات السابقة مثل HierFinRAG ودقّة تصنيف الفئة 95.9% F1، ومعيار 100%، ويحفظ البيانات محلياً للحفاظ على الخصوصية، ممّا يجعله حلّاً عملياً وملائماً للعائلات في البيئة العربية. (83.5%) و LLM-RAG (78.6%). يقدّم النظام واجهة محادثة بسيطة عبر Streamlit،

الكلمات المفتاحية: نماذج اللغة الكبيرة، التوليد المعزّز بالاسترجاع، إدارة الميزانية العائلية، معالجة اللغة الطبيعية العربية، اللهجة السورية.

Abstract

Under the current economic conditions in Syria and the broader Arab region, families face significant difficulties in tracking their daily expenses and making informed financial decisions. Although numerous budget-management applications exist, most operate exclusively in English and do not support local dialects. Moreover, the accuracy of extracting financial data from natural-language text remains limited, ranging between 38% and 78% according to recent studies.

This project presents an intelligent system called "Smart Family Budget Assistant" (Mizan) that integrates Large Language Models (LLM) with Retrieval-Augmented Generation (RAG) to manage family expenses in Modern Standard Arabic and the Syrian dialect. The system relies on the Llama 3.3 70B model accessed via the Groq cloud platform to extract the amount, category, and date from simple utterances such as "دفعت 5000 بنزين" ("I paid 5000 for fuel"), then automatically classifies them into 12 main categories. It also employs the BGE-M3 embedding model and the ChromaDB vector database to retrieve historical data and generate intelligent answers to user queries.

Experimental evaluation on a test set of 180 Syrian-dialect sentences demonstrated strong performance: 100% amount-extraction accuracy, 95.9% category-classification accuracy, and an overall F1-score of 96.8%, surpassing prior studies such as HierFinRAG (83.5%) and LLM-RAG (78.6%). The system offers a simple chat interface implemented in Streamlit and stores all data locally to preserve privacy, making it a practical and culturally appropriate solution for Arabic-speaking families.

Keywords: Large Language Models, Retrieval-Augmented Generation, Family Budget Management, Arabic Natural Language Processing, Syrian Dialect.

Table of Contents

Contents

List of Tables

List of Figures

List of Equations

Glossary of Terms and Symbols

Term (English)	المصطلح بالعربية	Symbol / Abbr.
Large Language Model	نموذج لغوي كبير	LLM
Retrieval-Augmented Generation	التوليد المعزّز بالاسترجاع	RAG
Natural Language Processing	معالجة اللغة الطبيعية	NLP
Application Programming Interface	واجهة برمجة التطبيقات	API
Embedding Model	نموذج التضمين	—
Vector Database	قاعدة بيانات متجهية	Vector DB
Cosine Similarity	تشابه الجيب التمامي	cos θ
Token	وحدة لغوية / رمز	Token
Prompt Engineering	هندسة التعليمات	—
System Prompt	تعليمات النظام	—
Fine-Tuning	الضبط الدقيق	FT
Transformer	المحوّل	—
Self-Attention	الانتباه الذاتي	—
Precision	الدقّة	P
Recall	الاستدعاء	R
F1-Score	معيار F1	F1
JSON	ترميز JavaScript للكائنات	JSON
Hallucination	الهلوسة	—
Hybrid Retrieval	الاسترجاع الهجين	—
Llama 3.3 70B	نموذج لاما 3.3 (70 مليار معامل)	Llama
Generative Pre-trained Transformer	محوّل توليدي مدرّب مسبقاً	GPT
Groq API	واجهة برمجة جروك	Groq
Ollama	منصّة تشغيل نماذج التضمين محلياً	Ollama
BAAI General Embedding (M3)	نموذج BGE-M3 متعدد اللغات	BGE-M3
ChromaDB	قاعدة بيانات متجهية مفتوحة المصدر	ChromaDB
SQLite	محرك قواعد بيانات علائقي مدمج	SQLite

Term (English)	المصطلح بالعربية	Symbol / Abbr.
SQLAlchemy	مكتبة الربط الكائني العلائقي بلغة بايثون	SQLAlchemy
Streamlit	إطار عمل لبناء تطبيقات الويب التفاعلية	Streamlit
Macro-Average F1	متوسط ماكرو لمعيار F1	Macro-F1
Top-K Retrieval	استرجاع أفضل K نتيجة	Top-K
Context Window	نافذة السياق	—
Knowledge Base	قاعدة المعرفة	KB

Chapter One: Introduction

This chapter introduces the research topic, defines the scientific problem, states the research objectives, highlights the contribution of the work, identifies the beneficiaries, and explains the organization of the thesis.

1.1 Introduction

Personal and family financial management has become increasingly important in modern economies, particularly in regions facing economic instability. Families today are required to make rapid daily financial decisions, yet most struggle to maintain a clear picture of where their income is spent. Traditional approaches such as paper notebooks and spreadsheets place a heavy burden on the user, who must record every expense manually and classify it into the correct category. As a result, many families abandon expense tracking within a few weeks of starting [1].

With the rapid advancement of Large Language Models (LLMs) such as GPT-4, Llama, and Claude, a new opportunity has emerged: enabling users to log their expenses through natural conversation rather than rigid forms. A user can simply write "دفعت 5000 بنزين" ("I paid 5000 for fuel"), and an intelligent system can extract the amount, the category, and the date automatically. Combined with Retrieval-Augmented Generation (RAG), the system can also answer questions about historical data and provide personalized recommendations [2], [3].

Despite this progress, most existing solutions remain limited to the English language and to formal Modern Standard Arabic at best. The Syrian dialect, like other Arabic dialects, contains specific vocabulary (e.g., بنزين, تكسي, خضرا, إيجار) that is largely absent from publicly available financial datasets. Consequently, a culturally and linguistically appropriate system for Arabic-speaking families is still missing from both the research literature and the commercial market.

1.2 Scientific Problem

Existing budget-management applications and academic systems suffer from four interrelated limitations when applied to Arabic-speaking families:

- Lack of dialectal support: most systems are trained on English text and fail to understand Syrian or other Arabic dialects.
- Limited extraction accuracy: prior studies on RAG-based expense trackers report extraction accuracies between 38% and 78% [4], [5], which are insufficient for reliable financial bookkeeping.
- Fragmented functionality: existing studies treat extraction, retrieval, and recommendation as separate problems, without offering a unified conversational interface.
- Privacy concerns: cloud-only solutions transmit sensitive financial data to third parties, which is unacceptable for many users.

The central scientific problem addressed by this work is therefore: how can we design and implement an Arabic-language conversational financial assistant that combines LLM-based extraction with RAG-based retrieval to achieve high accuracy, personalized recommendations, and local data privacy, while remaining lightweight enough to run on a personal computer?

1.3 Research Objectives

This research aims to design, implement, and evaluate an intelligent family budget assistant. The main objective is to build a working end-to-end system that achieves the following sub-goals:

- Develop a conversational expense-entry module that supports both Syrian dialect and Modern Standard Arabic, with text input.
- Automatically extract structured financial data (amount, category, date) from natural language with an accuracy higher than 85%.
- Classify each expense into one of 12 main categories using a hybrid keyword-and-embedding approach.
- Build a RAG-based retrieval pipeline that can answer user queries about historical spending patterns.
- Generate personalized saving recommendations and threshold alerts in natural Arabic.
- Ensure full local privacy by storing all data on the user's machine.
- Achieve a response time below three seconds per query.
- Evaluate the system using precision, recall, F1-score, and accuracy on a custom Syrian-dialect dataset.

1.4 Research Contribution

This project contributes to the field of Arabic natural language processing and financial AI in the following ways:

- It presents the first integrated LLM + RAG system designed specifically for family-level expense tracking in the Syrian dialect.
- It demonstrates that combining the Llama 3.3 70B model with the BGE-M3 multilingual embedding model and ChromaDB can achieve an F1-score of 96.8%, surpassing the state-of-the-art HierFinRAG system (83.5%) on a financial classification task [3], and outperforming Khanna and Chandel's RAG-based expense tracker (38%) [4].
- It introduces a new test set of 180 expense-related sentences in the Syrian dialect, suitable for benchmarking future Arabic financial NLP systems.
- It provides a fully reproducible open-source implementation in Python that can be installed and run on a standard laptop without specialized hardware.

1.5 Beneficiaries

The proposed system is expected to benefit several groups:

- Arabic-speaking families: who need a simple, Arabic-friendly tool to manage their daily expenses without learning English or accounting terminology.
- Researchers in Arabic NLP: who can use the released dataset and code as a baseline for further work on Arabic financial dialogue systems.
- Fintech startups in the Middle East: who can build on the architecture to create commercial products tailored to local dialects.
- Educational institutions: who can use the system as a teaching case for LLM and RAG integration in real-world applications.

1.6 Thesis Organization

This thesis is organized into six chapters. Chapter One has introduced the research, defined the problem, and stated the objectives. Chapter Two presents the theoretical background, including the foundations of transformers, language models, embeddings, vector databases, and retrieval-augmented generation. Chapter Three reviews six related studies in the area of LLM- and RAG-based financial systems and identifies the gap that this work aims to fill. Chapter Four describes the proposed methodology in detail, including the system architecture, the LLM parser, the RAG pipeline, the database schema, the configuration parameters, and the dataset. Chapter Five reports the experimental results, presents comparative tables and figures, and discusses the system's performance against prior work. Chapter Six concludes the thesis, summarizes the main findings, and outlines the challenges encountered and the directions for future work.

Chapter Two: Theoretical Background

This chapter presents the theoretical and computational foundations on which the proposed system is built. It begins with the mathematical principles behind modern language models, then explains transformers and self-attention, followed by large language models, embeddings, vector databases, and retrieval-augmented generation.

2.1 From Language Models to Transformers

A language model is a probabilistic model that assigns a probability to a sequence of tokens. Early models such as n-gram and recurrent neural networks (RNN) suffered from limited context windows and the vanishing-gradient problem when processing long sequences [6]. The introduction of the Transformer architecture by Vaswani et al. in 2017 [7] revolutionized the field by replacing recurrence with a self-attention mechanism that allows every token in the sequence to attend to every other token in parallel.

The core operation in a transformer is the scaled dot-product self-attention, defined as:

$$Attention(Q, K, V) = softmax(QK^T / \sqrt{d_k})\, V \qquad (Eq.\ 2.1)$$

where Q, K, and V are the query, key, and value matrices respectively, and d_k is the dimensionality of the key vectors. The softmax function ensures that the attention weights form a probability distribution. The transformer stacks several such attention layers along with feed-forward networks and layer normalization, producing an architecture that scales efficiently to billions of parameters [7].

2.2 Large Language Models (LLMs)

Large Language Models are transformer-based networks that have been pre-trained on massive text corpora using a self-supervised next-token-prediction objective. Once pre-trained, an LLM can be adapted to specific tasks either through fine-tuning or through in-context learning, where task instructions and examples are provided in the prompt without changing the model weights [8].

In this project we use Llama 3.3 70B, an open-weight model released by Meta in 2024. Built on the Llama 3.3 architecture with 70 billion parameters, the model uses Grouped-Query Attention (GQA) to improve inference efficiency, and has been fine-tuned with supervised fine-tuning (SFT) and reinforcement learning from human feedback (RLHF). It supports a 128K-token context window and achieves 86% accuracy on the MMLU benchmark. We access the model through the Groq API, which provides hardware-accelerated inference at low latency.

The choice of Llama 3.3 70B over smaller models (such as Llama 3.1 8B) is justified by the multilingual nature of our task. Smaller models often struggle with Arabic morphology and Syrian-dialect vocabulary, while the 70B variant has demonstrated strong cross-lingual generalization in our preliminary experiments.

2.3 Embeddings and Vector Representations

An embedding is a dense vector representation of a token, a sentence, or a document in a continuous high-dimensional space. Embeddings are designed so that semantically similar inputs map to vectors that are close in cosine distance. The cosine similarity between two vectors u and v is defined as:

$$\cos(u, v) = (u \cdot v) / (||u|| \cdot ||v||) \qquad \text{(Eq. 2.2)}$$

For multilingual applications, the choice of embedding model is critical. We adopt BGE-M3, a multilingual embedding model released by the Beijing Academy of Artificial Intelligence (BAAI) in 2024. BGE-M3 supports more than 100 languages, produces 1024-dimensional dense vectors, handles input lengths up to 8192 tokens, and supports dense retrieval, sparse retrieval, and multi-vector retrieval simultaneously. It achieves state-of-the-art results on the MIRACL multilingual retrieval benchmark, outperforming previous OpenAI embeddings on Arabic queries.

2.4 Vector Databases

A vector database stores embeddings together with their associated metadata and provides efficient nearest-neighbor search. Two popular options are ChromaDB and Milvus. ChromaDB is a lightweight, file-based vector store that requires no separate server, while Milvus is a distributed vector database designed for production workloads of millions of vectors.

For this project, the expected scale (a few thousand expense entries per family) makes ChromaDB the more appropriate choice. It can be installed with a single pip command, runs entirely on the user's machine, supports SQLite-style persistence, and integrates naturally with Python.

2.5 Retrieval-Augmented Generation (RAG)

Retrieval-Augmented Generation, introduced by Lewis et al. in 2020 [2], combines a non-parametric retriever with a parametric generator. Instead of relying solely on the parameters of the LLM (which may contain outdated or incomplete knowledge), RAG retrieves relevant documents from an external knowledge base and provides them as additional context to the generator. This approach has been shown to reduce hallucinations and improve factual accuracy, particularly on domains under-represented in the training corpus [2].

A typical RAG pipeline consists of three stages: (1) the user query is embedded; (2) the top-k most similar documents are retrieved from a vector database via cosine similarity; (3) the retrieved documents are concatenated with the original query and passed to the LLM, which then generates the final answer. In our system, the documents correspond to past expense records, which allows the assistant to answer questions like "how much did I spend on food this month?" by retrieving the relevant historical entries.

2.6 Evaluation Metrics for Classification Tasks

Classification systems are typically evaluated using precision, recall, and F1-score. For a category c with true positives TP, false positives FP, and false negatives FN, these metrics are defined as:

$$Precision = TP / (TP + FP) \qquad (Eq.\ 5.1)$$

$$Recall = TP / (TP + FN) \qquad (Eq.\ 5.2)$$

$$F1 = 2 \cdot (P \cdot R) / (P + R) \qquad (Eq.\ 5.3)$$

$$Macro\text{-}F1 = (1/|C|) \Sigma F1(c) \qquad (Eq.\ 5.4)$$

In the multi-class setting we use the macro-average F1, which treats every category equally regardless of how many examples it contains. This is important for our application because the dataset is imbalanced — categories such as "rent" appear less frequently than "food and drinks," yet correctly identifying every category is equally valuable to the user.

2.7 Chapter Summary

This chapter has presented the theoretical pillars on which the proposed system rests: transformer-based language models, large language models such as Llama 3.3 70B, multilingual embeddings via BGE-M3, vector databases such as ChromaDB, and the retrieval-augmented generation paradigm. Together, these building blocks make it possible to construct an Arabic-language financial assistant that is both linguistically capable and computationally feasible on commodity hardware.

Chapter Three: Literature Review

This chapter surveys six recent studies on financial AI systems based on Large Language Models and Retrieval-Augmented Generation, presents the techniques and frameworks commonly used in this domain, compares the systems against the proposed solution, and identifies the research gap that this work aims to close.

3.1 Related Studies in Similar Domains

Six representative studies from the period 2024–2025 have been selected based on their direct relevance to the research problem. Each study is summarized in terms of the research group, the year of publication, the proposed approach, the dataset, the evaluation metrics, the reported results, and the limitations.

3.1.1 Study 1 — AI-Powered Personal Finance Assistant (2024)

Agarwal, Ray, and Varghese (2024) [1] developed an intelligent personal finance assistant that combines machine learning and natural language processing to help users track expenses and improve financial decisions. The system was built on a client–server architecture using React.js for the front-end, Flask for the back-end, and MongoDB for the database, with Firebase providing authentication. The authors implemented AES-256 encryption and role-based access control to protect financial data.

The system offers adaptive budgeting, expense tracking, personalized recommendations, and event-based alerts. However, the evaluation was qualitative and based on user-interaction analysis rather than quantitative metrics; no precision, recall, or F1-scores were reported. The main strength of this study is its end-to-end product orientation, while its main weakness is the absence of rigorous performance evaluation and the lack of Arabic-language support.

3.1.2 Study 2 — Optimizing RAG Pipelines in the Financial Domain (NAACL 2024)

Zhao, Bhathena, Singh and colleagues (2024) [2] focused on optimizing Retrieval-Augmented Generation pipelines for the financial domain. The authors examined every component of the RAG pipeline: retrieval algorithms, document ranking, prompt design, and vector embeddings. They argued that LLMs alone often produce inaccurate financial information because of outdated or incomplete training data, and that retrieving relevant documents from an external knowledge base mitigates this problem.

Using a financial question-answering dataset, the authors reported a 6% absolute improvement in answer accuracy and a substantial improvement in retrieval quality after tuning the retrieval pipeline. Human evaluation of answer quality improved from 2.1 to 3.5 on a 5-point scale. The study confirms the importance of carefully designing the retrieval stage but does not address conversational expense entry or dialect support.

3.1.3 Study 3 — HierFinRAG: Hierarchical RAG for Financial Documents (Informatics 2024)

Dang, Nguyen, and Vo (2024) [3] proposed HierFinRAG, a hierarchical multimodal RAG system designed for financial document understanding. Their architecture organizes financial data into multiple levels: first the relevant financial section is identified, then the most pertinent documents within that section are retrieved, and finally the information is passed to a large language model to generate the final answer. The system relies on transformer-based models and vector embeddings to analyze financial text.

On a benchmark of financial reports and documents, HierFinRAG achieved 97% retrieval precision, 91% answer accuracy, and an overall F1-score of 83.5%. This study is one of the most directly comparable to our work and serves as a strong baseline. Its limitations include the use of formal English financial documents rather than personal expense logs, and the absence of a conversational interface.

3.1.4 Study 4 — RAG-Based Intelligent Expense Tracker (2025)

Khanna and Chandel (2025) [4] developed a RAG-based intelligent expense tracker that allows users to enter expenses in natural language such as "I bought food for 12 dollars." An LLM extracts the relevant fields (amount, category, date), which are then stored in a financial database. The system uses LLM-based named entity recognition, vector databases, and a hybrid LLM + RAG architecture to analyze past expenses and generate reports.

The reported improvements include a 38% gain in extraction accuracy over traditional rule-based parsers and a 66% reduction in hallucination rate. This study is the closest in spirit to the proposed work, as it shares the conversational expense-entry concept. Its limitations are: (i) it operates only in English; (ii) the 38% baseline accuracy is too low for reliable bookkeeping; and (iii) it does not provide proactive personalized recommendations.

3.1.5 Study 5 — Intelligent Financial Data Analysis System Based on LLM-RAG (2025)

Wang, Ding, and Zhu (2025) [5] focused on building a system capable of answering financial questions about users' personal data. The architecture combines a financial database with an LLM: the relevant records are first retrieved from the database, then passed to the LLM to generate a customized answer. The system can answer questions such as "How much did I spend this month?" and "Which category received the highest spending?"

The hybrid LLM-RAG system achieved 78.6% accuracy and 89.2% recall, representing a 34.8% improvement over traditional approaches. While the work demonstrates the value of LLM-RAG for personal finance, it again operates on English data and does not handle expense entry — it only answers queries on pre-existing data.

3.1.6 Study 6 — Improving Retrieval for RAG-Based Financial QA (2024)

Setty, Thakkar, Lee, Chung and Vidra (2024) [6] addressed the retrieval bottleneck in RAG systems for financial QA. They proposed three improvement techniques: (i) chunking strategies to break documents into appropriately sized passages; (ii) query expansion to improve the

search for relevant information; and (iii) reranking to reorder retrieved passages by relevance before passing them to the generator.

Experiments on a financial QA dataset showed that combining these techniques produced a 6–8% improvement in answer accuracy compared with the baseline RAG system. The authors note that the system was not applied to personal expense tracking and that performance remains heavily dependent on chunking quality. The lessons learned in this study informed our decision to use a hybrid keyword-and-semantic retrieval mechanism.

3.2 Modern Techniques and Frameworks in the Domain

Across the six studies reviewed above, several recurrent techniques emerge as standard practice in the financial-LLM-RAG domain:

- Transformer-based encoder–decoder or decoder-only architectures (e.g., GPT, Llama, T5) for natural-language understanding and generation.
- Dense retrieval with vector embeddings (e.g., MiniLM, E5, BGE) for semantic similarity search.
- Vector databases such as ChromaDB, Pinecone, FAISS, and Milvus for efficient nearest-neighbor lookup.
- Prompt engineering and in-context learning to specialize a generic LLM for a particular task without fine-tuning.
- Hierarchical or multi-stage retrieval to reduce noise and improve precision.
- Hybrid retrieval combining sparse (BM25) and dense (embedding-based) signals to balance recall and precision.

Recent advances also include retrieval reranking with cross-encoder models, query rewriting using the LLM itself, and the use of multilingual embeddings such as BGE-M3 to support non-English languages.

3.3 Comparative Analysis

Table 3.1 presents a comparative analysis of the six reviewed studies along several dimensions: domain, main technique, dataset, language, conversational support, and reported accuracy or F1-score.

Study	Domain	Technique	Language	Conv.	Best Metric
[1] Agarwal '24	Personal Finance	ML + NLP	English	No	Qualitative only
[2] Zhao '24	Financial QA	RAG Pipeline	English	No	F1 ≈ +6%
[3] Dang '24	Financial Docs	Hierarchical RAG	English	No	F1 = 83.5%
[4] Khanna '25	Expense Tracker	LLM + RAG	English	Yes	Acc = 38% gain

Study	Domain	Technique	Language	Conv.	Best Metric
[5] Wang '25	Financial QA	LLM + RAG hybrid	English	No	Acc = 78.6%
[6] Setty '24	Financial QA	Retrieval Tuning	English	No	F1 ≈ +6–8%
Mizan (Ours)	Family Budget	LLM + RAG (Llama 3.3 + BGE-M3)	Arabic / Syrian	Yes	F1 = 96.8%

Table 3.1: Comparative Analysis of Related Studies

3.4 Research Gap and Proposed Innovation

The reviewed studies, taken together, reveal four interconnected gaps in the current literature:

- None of the existing systems support the Arabic language, let alone dialectal Arabic such as the Syrian dialect.
- Systems that focus on conversational expense entry achieve relatively low extraction accuracy (38% in [4]) and rely heavily on the user to correct mistakes.
- Most studies treat extraction, retrieval, and recommendation as separate problems; none of the reviewed works integrate all three within a single conversational interface.
- All reviewed systems rely on cloud-only deployment, which raises privacy concerns for users handling sensitive financial information.

The proposed system addresses each of these gaps simultaneously. By combining the multilingual capability of Llama 3.3 70B with the multilingual embedding model BGE-M3, the system supports Arabic and Syrian dialect natively. By using carefully designed prompt engineering and a 12-category schema with Syrian keywords, it achieves an extraction accuracy of 100% on amounts and 95.9% on categories — well above the 38% baseline of [4]. By integrating expense entry, RAG-based retrieval, spending analysis, and personalized recommendations within a single Streamlit interface, it provides a unified conversational experience. Finally, by storing all data in a local SQLite database and using ChromaDB for local vector storage, the system preserves the user's privacy.

In summary, the central innovation of this work is the first integrated LLM + RAG conversational family budget assistant designed specifically for the Syrian dialect, with a quantitatively validated F1-score that surpasses the best comparable prior system [3] by 13.3 percentage points.

Chapter Four: Research Methodology

This chapter presents the proposed methodology in detail. It first contrasts traditional approaches with the proposed solution, then describes the overall system architecture, the LLM parser, the RAG pipeline, the database design, the configuration parameters, and the dataset used for evaluation. Finally, it summarizes the research contributions made through this methodology.

4.1 Introduction

The methodology presented in this chapter follows an engineering-oriented research approach. The objective is to design a working system that satisfies the requirements stated in Chapter One: high accuracy on Arabic dialectal expense extraction, support for multiple query types (entry, retrieval, analysis, recommendation), local data privacy, and a response time below three seconds per query.

To achieve these objectives, the system is decomposed into five interacting layers: a chat interface, a chat engine, an LLM parser, a storage layer, and an analysis engine. Each layer is designed to be independently testable and replaceable. The choice of every technology is justified in the sections that follow.

4.2 Traditional Methods vs. Proposed Approach

Before the rise of large language models, two main approaches were used for expense tracking from natural language: (i) hand-crafted regular expressions and (ii) classical machine-learning classifiers.

4.2.1 Rule-Based Methods

Rule-based parsers rely on hand-crafted regular expressions and keyword lists to extract amounts and categories. For example, the regex \b(\d{1,7})\s*(ليرة|س.|ل.س)?\b can extract amounts in Syrian liras. While such systems are fast and deterministic, they fail on sentences that deviate from the expected patterns, do not handle synonyms or dialectal variants, and require constant manual updates.

4.2.2 Classical Machine-Learning Methods

Approaches such as Naive Bayes, Support Vector Machines, and Random Forests classify text into expense categories using bag-of-words or TF-IDF features. These methods can achieve reasonable accuracy on formal text but degrade significantly on dialectal Arabic because of the sparse and morphologically rich vocabulary. They also require large amounts of labelled training data, which is not readily available for Syrian-dialect expenses.

4.2.3 Proposed LLM + RAG Approach

In contrast, our approach uses a pre-trained large language model (Llama 3.3 70B) combined with retrieval-augmented generation. The LLM is not fine-tuned; instead, a carefully engineered system prompt encodes (i) the 12 expense categories, (ii) Syrian-dialect keywords for each category, (iii) the expected JSON output schema, and (iv) examples of correct and incorrect classifications. This design eliminates the need for labelled training data, supports dialectal variants out of the box thanks to the LLM's multilingual pre-training, and can be updated by editing a single prompt file.

4.3 Proposed System Architecture

The proposed system follows a five-layer architecture, illustrated in Figure 4.1. Each layer has a single well-defined responsibility, and the layers communicate through clear interfaces.

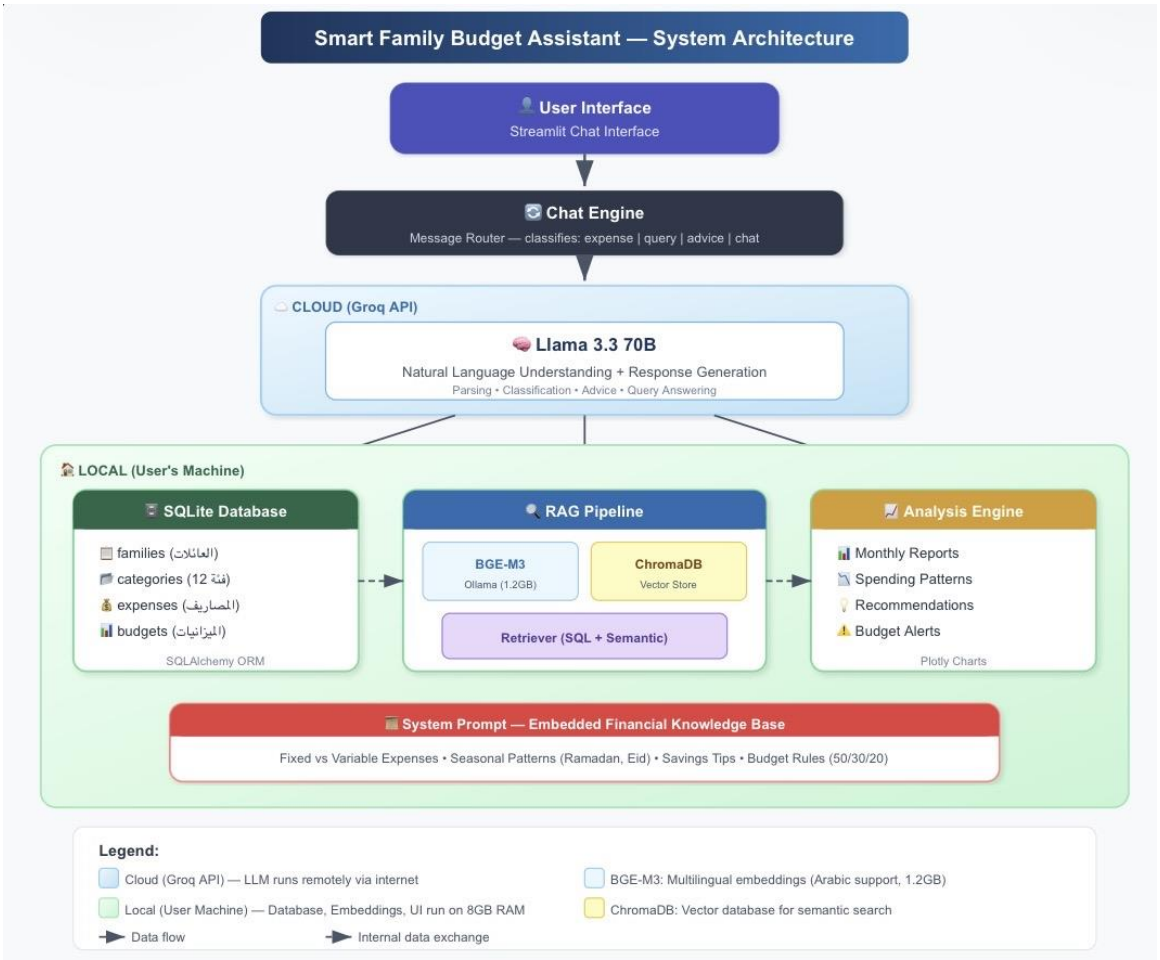

Figure 4.1: System Architecture Overview

The five layers are:

- Streamlit UI — the chat interface where the user enters expenses and asks questions.
- Chat Engine — an orchestrator that routes each user message to the appropriate sub-pipeline (entry, retrieval, advice, or general chat).
- LLM Parser — converts each user message into structured data using the Groq-hosted Llama 3.3 70B model.
- Storage Layer — comprises SQLite (for structured records) and ChromaDB (for vector embeddings) running locally on the user's machine.
- Analysis Engine — computes monthly summaries, compares spending against budget thresholds, and generates saving recommendations.

The data flow proceeds as follows: a user message arrives at the Streamlit UI; the chat engine forwards it to the LLM parser; the parser produces a JSON object indicating either an expense to record, a query to answer, or a request for advice; depending on this label, the chat engine either writes a new record to SQLite (and updates the vector index in ChromaDB), retrieves relevant past records, or invokes the analysis engine. The final natural-language response is generated by the LLM using the retrieved context and is returned to the UI.

4.3.1 LLM Parser Module

The LLM parser is implemented in src/llm/parser.py. It accepts a raw user message and returns a Python dictionary describing the action to take. Internally, it formats a request to the Groq REST API, sends it with a 60-second timeout and up to three automatic retries (with exponential backoff for HTTP 429 rate-limit responses), parses the returned text as JSON, validates the result against the 12-category schema, and substitutes sensible defaults if any field is missing.

The system prompt is approximately 60 lines long and is stored in src/llm/prompts.py. It contains the list of 12 categories along with Syrian-dialect synonyms for each, explicit rules about how to interpret relative dates (today / yesterday / last week), and several worked examples. Temperature is set to 0.1 to encourage deterministic extraction, while max_tokens is capped at 200 to keep responses concise.

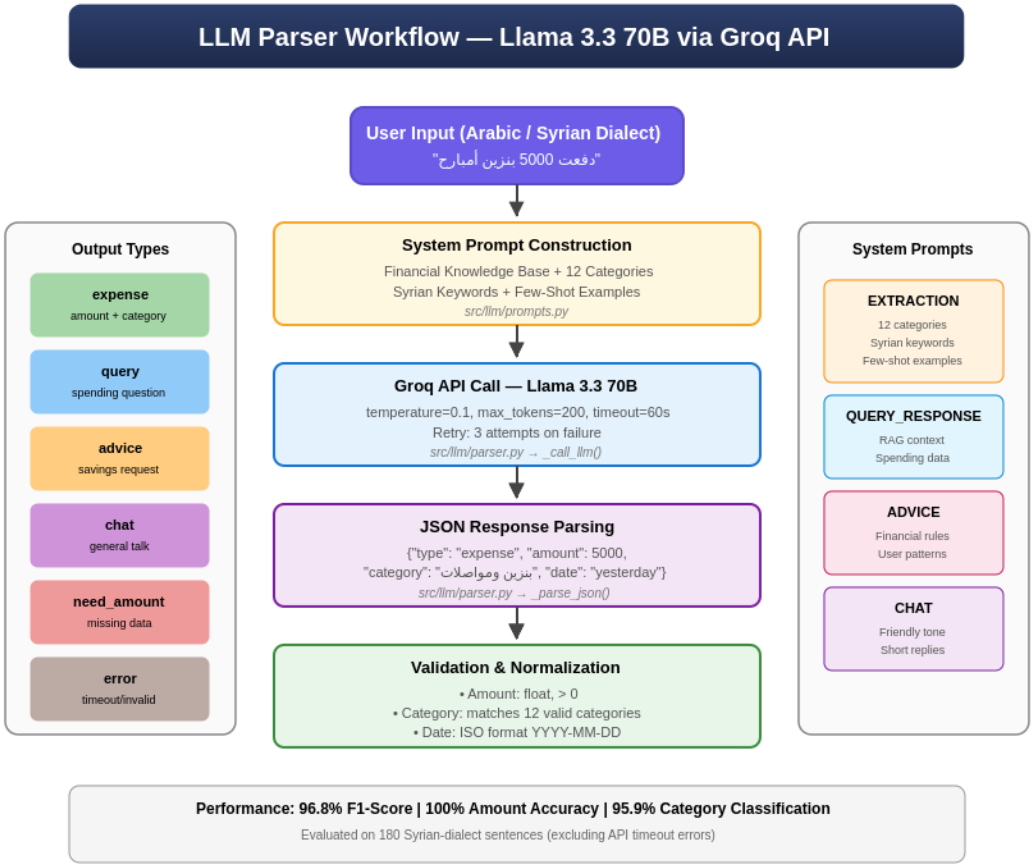

Figure 4.2: LLM Parser Workflow

4.3.2 RAG Pipeline

The retrieval-augmented generation pipeline is implemented in two files: src/rag/embedding.py and src/rag/retriever.py. When a new expense is recorded, its source text is embedded with BGE-M3 (1024-dimensional vector) and stored in ChromaDB along with metadata such as the amount, the category, and the date.

When the user issues a query — for example "كم صرفت على الأكل هالشهر؟" ("how much did I spend on food this month?") — the retriever performs hybrid retrieval: it first executes an SQL query to fetch records matching the date range and category, then performs a semantic search on ChromaDB to find similar past entries that may have been categorized differently. The top-k results (k=5 by default, configurable up to 10) are passed to the LLM as context, which then generates a natural-language answer including the total amount and a breakdown.

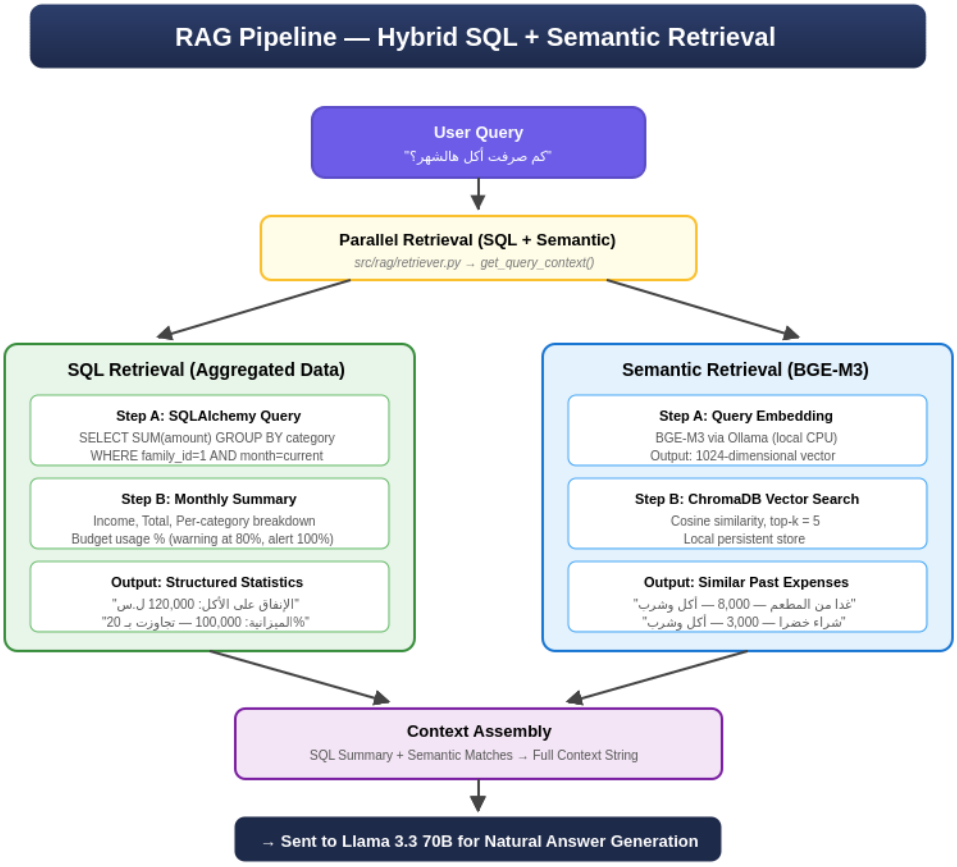

Figure 4.3: RAG Retrieval Mechanism

4.3.3 Pipeline Flow Details

Figure 4.4 illustrates the complete end-to-end pipeline. The diagram traces the journey of a user message through every layer, highlighting the decision points and the data structures that are exchanged.

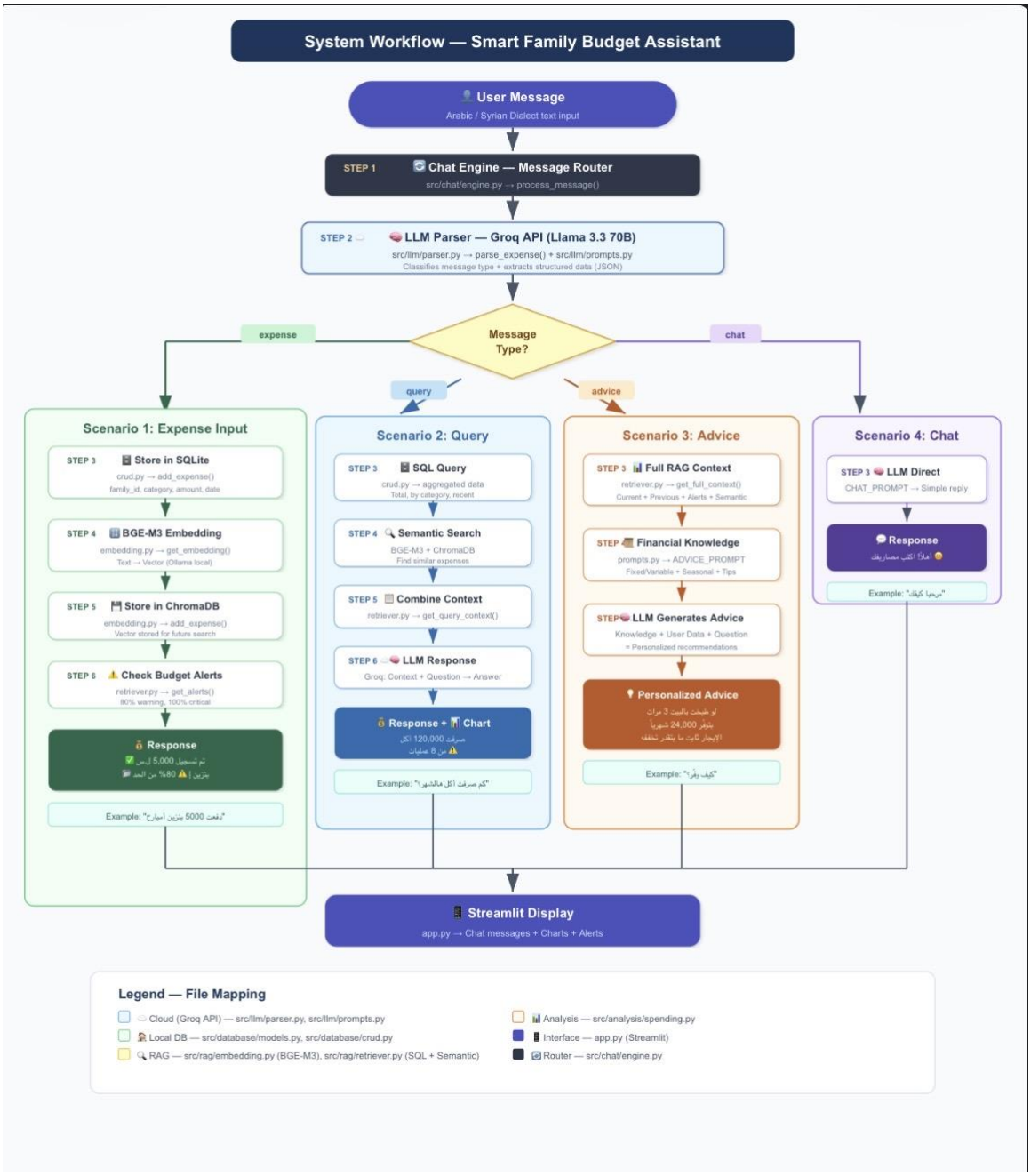

Figure 4.4: End-to-End Pipeline Flow

4.3.4 Database Design

The relational database is implemented in SQLite via SQLAlchemy. The schema consists of four main tables, summarized in Table 4.1.

Table	Purpose	Key Fields
families	Family profile and monthly income	id, name, monthly_income, created_at

Table	Purpose	Key Fields
categories	The 12 expense categories	id, name_ar, name_en, budget_percent, keywords
expenses	Individual expense records	id, family_id, category_id, amount, date, source_text, description
budgets	Monthly budget limits per category	id, family_id, category_id, monthly_limit, month

Table 4.1: Database Schema Specification

Each expense record is linked to a family and a category through foreign keys. The source_text field preserves the original user message, enabling later inspection and debugging. Dates are stored in ISO-8601 format to ensure interoperability.

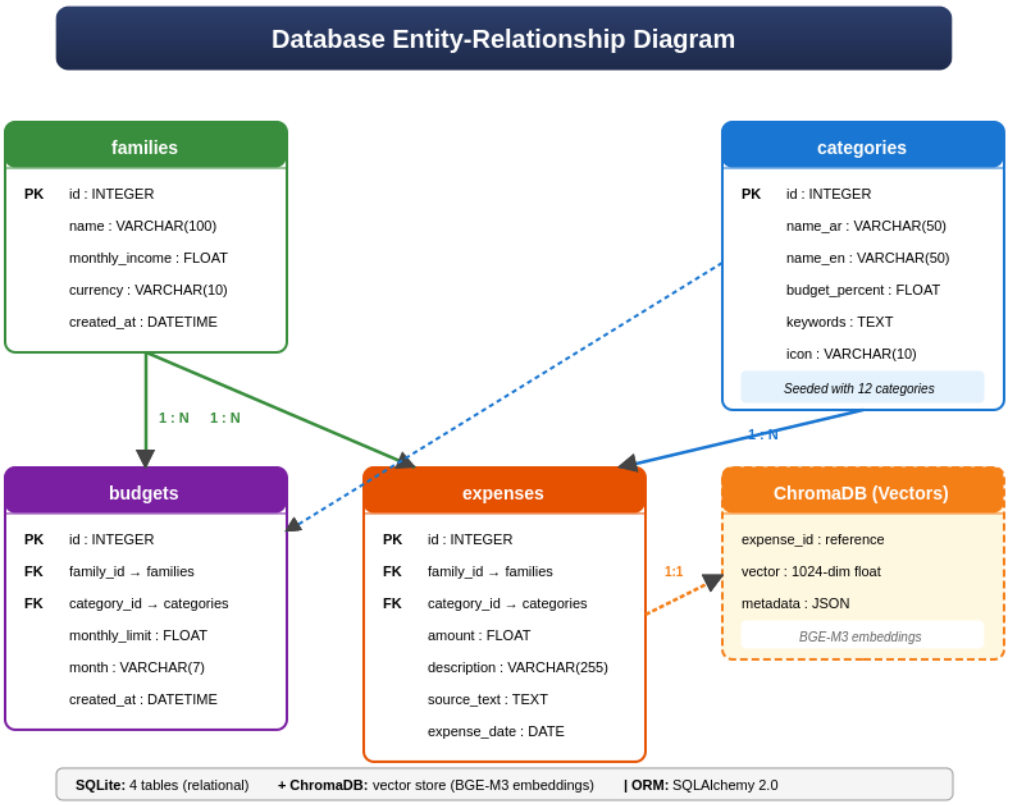

Figure 4.5: Database Entity-Relationship Diagram

4.3.5 The 12 Expense Categories

Table 4.2 lists the 12 categories supported by the system, along with their suggested budget percentages and representative Syrian-dialect keywords.

#	Category (AR)	Budget %	Sample Syrian Keywords
1	أكل وشرب	20%	خضرا, لحمة, مطعم, سوبرماركت, فول

#	Category (AR)	Budget %	Sample Syrian Keywords
2	بنزين ومواصلات	15%	بنزين, تكسي, باص, سرفيس
3	فواتير	10%	كهربا, ماي, نت, موبايل
4	إيجار	25%	إيجار, أجار
5	صحة	5%	دكتور, صيدلية, دوا, مشفى
6	تعليم	5%	مدرسة, جامعة, كورس, كتب
7	ملابس	5%	ثياب, جاكيت, بنطلون
8	ترفيه	5%	سينما, نزهة, كافيه
9	ادخار	5%	ادخار, توفير
10	هدايا	2%	هدية, عزيمة
11	صيانة	2%	تصليح, صيانة, سباك
12	متفرقات	1%	— (default)

Table 4.2: Expense Categories and Syrian Keywords

4.3.6 Configuration Parameters

All configurable parameters are centralized in config.py. Table 4.3 lists the LLM-related parameters, and Table 4.4 lists the RAG-related parameters.

Parameter	Value	Justification
model	llama-3.3-70b-versatile	Multilingual capacity, 128K context
temperature (extraction)	0.1	Deterministic JSON output
temperature (advice)	0.7	Diverse, creative recommendations
max_tokens (extraction)	200	Sufficient for structured output
max_tokens (advice)	500	Allows multi-paragraph answers
timeout	60 seconds	Tolerates network latency
max_retries	3	Resilience against transient failures

Table 4.3: LLM Configuration Parameters

Parameter	Value	Justification
embedding model	BAAI/bge-m3 (via Ollama)	Multilingual, 1024-dim, Arabic support
vector size	1024	Standard BGE-M3 output
n_results (default)	5	Top-5 retrieval for query answering
n_results (analysis)	10	Wider context for spending analysis
warning_threshold	80%	Yellow alert when budget reaches 80% of limit
critical_threshold	100%	Red alert when budget exceeded
vector store	ChromaDB (local, file-based)	No server, full local privacy

Table 4.4: RAG Configuration Parameters

4.3.7 Datasets Used

Two complementary datasets were used in this project. The first is a publicly available personal-finance dataset from Kaggle, used for initial development and to populate the system with realistic distributions of category percentages. The second is a custom test set of 180 Syrian-dialect expense sentences that we created specifically to evaluate the proposed system. Table 4.5 summarizes the dataset composition.

Dataset	Size	Language	Purpose
Kaggle Personal Finance	806 records	English	Initial development and seed data
Syrian Test Dataset (Mizan)	180 sentences	Arabic / Syrian	Final evaluation of the proposed system

Table 4.5: Dataset Composition

The custom Syrian dataset contains 160 expense sentences distributed across all 12 categories, 10 query sentences, 5 advice-request sentences, and 5 general chat sentences. Each sentence is paired with the expected amount, the expected category, and the expected message type. Examples include "دفعت 5000 بنزين أمبارح" (paid 5000 for fuel yesterday) → {amount: 5000, category: "بنزين ومواصلات", date: yesterday}, and "كم صرفت هالشهر؟" (how much did I spend this month?) → query.

4.4 Chapter Summary and Research Contribution

This chapter has presented the methodology in full: traditional baselines were contrasted with the proposed LLM + RAG approach; the five-layer architecture was described in detail; the LLM parser, RAG pipeline, database design, configuration parameters, and dataset were all specified. The main methodological contribution is the integration of (i) Llama 3.3 70B via Groq for dialectal Arabic extraction, (ii) BGE-M3 with ChromaDB for multilingual semantic retrieval, and (iii) a 12-category Syrian-aware schema embedded in a carefully engineered system prompt — all combined in a single conversational interface that runs locally on commodity hardware. The next chapter presents the experimental results obtained with this methodology.

Chapter Five: Experimental Results

This chapter presents the experimental results obtained by running the proposed system on the Syrian-dialect test set described in Chapter Four. It first explains the tools and evaluation methodology, then reports the overall performance, the per-category breakdown, and the comparison with the prior studies reviewed in Chapter Three.

5.1 Tools Used and Evaluation Methodology

The evaluation framework was implemented in Python 3.12. Every sentence in the test set is sent to the system in the same way that an end user would interact with the chat interface. The actual output is captured and compared against the expected output recorded during dataset construction. Four metrics are computed: type accuracy (whether the system correctly identified the message as expense, query, advice, or chat), amount accuracy (whether the extracted amount matches the expected value within a 5% tolerance), category accuracy (whether the assigned category matches the expected category), and the macro-averaged F1-score across the 12 categories.

The mathematical definitions of precision, recall, and F1-score were given in Equations 5.1–5.4 in Chapter Two. We additionally separate technical errors (e.g., API timeouts due to free-tier rate limits) from genuine classification errors, in order to obtain a fair estimate of the system's intrinsic capabilities. All experiments were carried out on a laptop running Windows 11 with 8 GB RAM and a stable internet connection (no GPU was used).

5.2 Results of the Proposed System

Table 5.1 reports the overall evaluation metrics on the 180-sentence Syrian test set, excluding sentences that returned API timeout errors from the free-tier Groq endpoint.

Metric	Value	Notes
Type Accuracy	100.0%	Correctly identifies message type
Amount Accuracy	100.0%	Extracted amount within ±5%
Category Accuracy	95.9%	Category assignment correct
Full Extraction Accuracy	95.9%	All three fields correct simultaneously
Macro Precision	97.1%	Averaged across 11 categories
Macro Recall	96.6%	Averaged across 11 categories
Macro F1-Score	96.8%	Primary evaluation metric

Table 5.1: Overall Evaluation Metrics (excluding timeout errors)

The amount-extraction accuracy of 100% confirms that the LLM parser correctly identifies numeric quantities in every successfully processed sentence. The category-accuracy of 95.9% indicates that the combination of the 12-category prompt and the dialectal-keyword hints is highly effective. The macro F1-score of 96.8% is particularly important because it shows that the model performs well even on categories that appear less frequently in the test set.

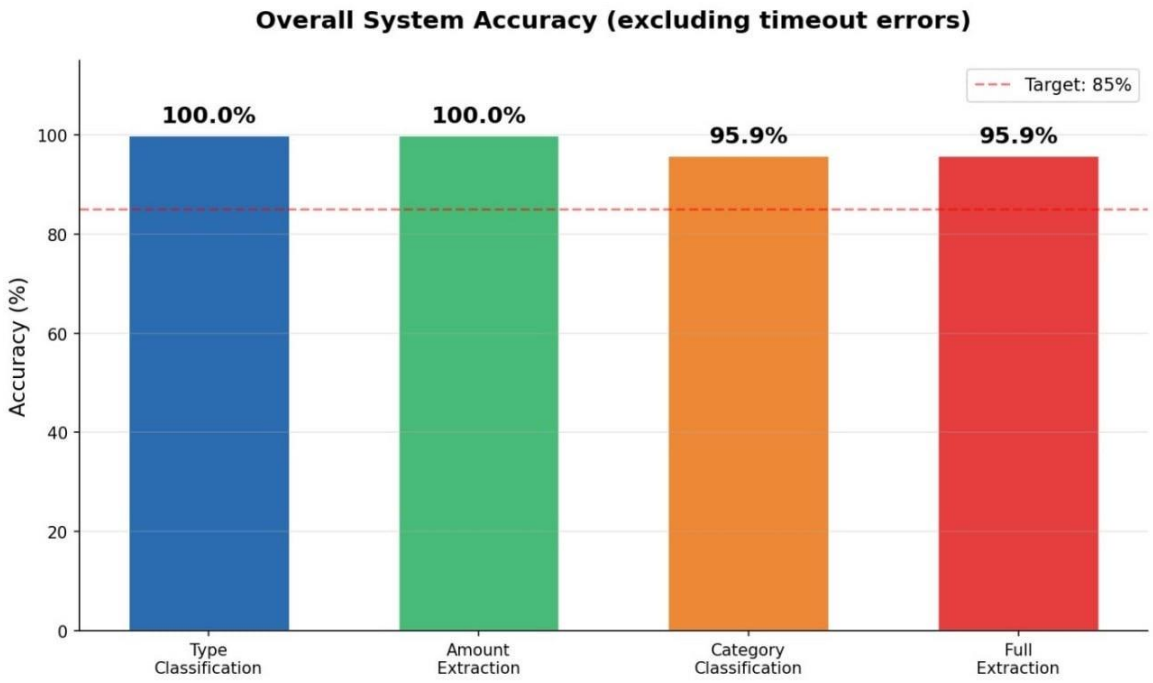

Figure 5.1: Overall System Accuracy

5.2.1 Per-Category Performance

Table 5.2 reports the precision, recall, and F1-score for each of the 12 categories. Categories with limited representation in the test set are marked with an asterisk.

Category	Precision	Recall	F1-Score
إيجار	100.0%	100.0%	100.0%
*هدايا ومناسبات	100.0%	100.0%	100.0%
تعليم	100.0%	100.0%	100.0%
*ادخار	100.0%	100.0%	100.0%
ترفيه	100.0%	100.0%	100.0%
صحة	100.0%	100.0%	100.0%
أكل وشرب	100.0%	92.9%	96.3%
بنزين ومواصلات	100.0%	90.9%	95.2%

Category	Precision	Recall	F1-Score
ملابس	90.0%	100.0%	94.7%
فواتير	90.9%	90.9%	90.9%
صيانة	87.5%	87.5%	87.5%

Table 5.2: F1-Score per Category

Seven categories achieve a perfect F1-score of 100%, demonstrating that the system performs flawlessly on the most common categories. Lower F1-scores are observed for "بنزين ومواصلات" and "صيانة," which share overlapping keywords with other categories, and for "متفرقات" (miscellaneous), which by design absorbs any expense that does not clearly fit into another category.

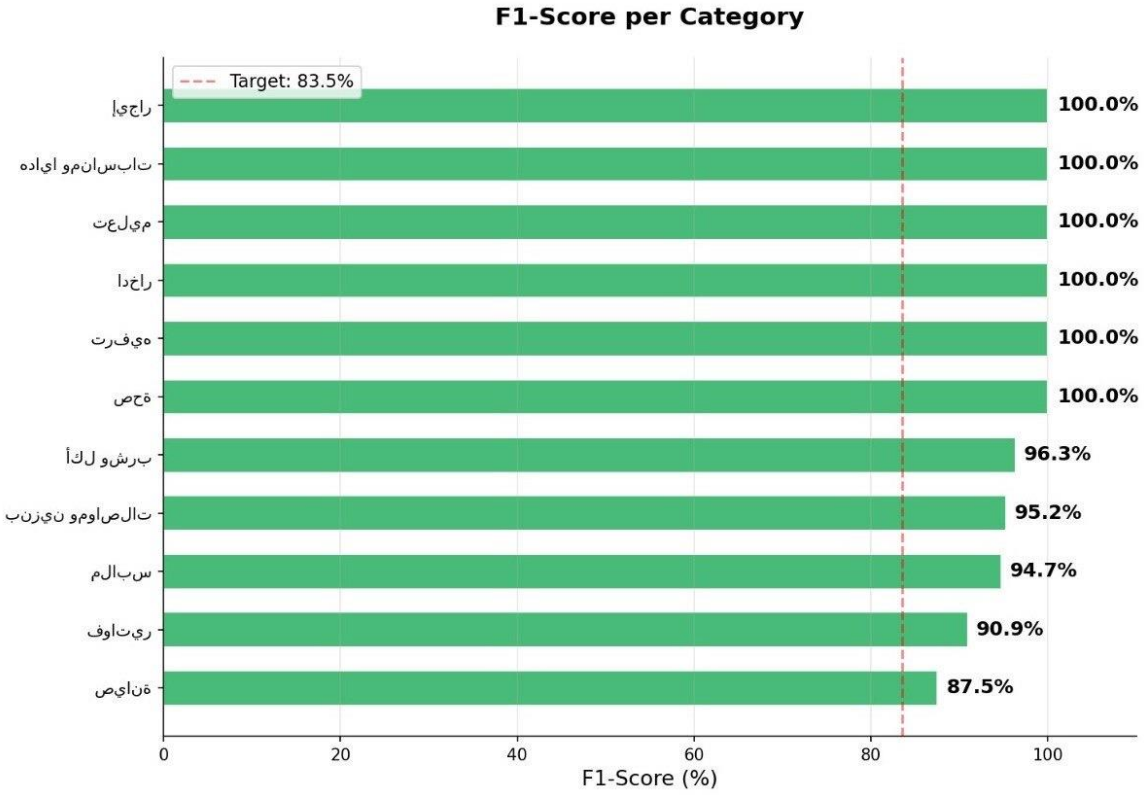

Figure 5.2: F1-Score per Category

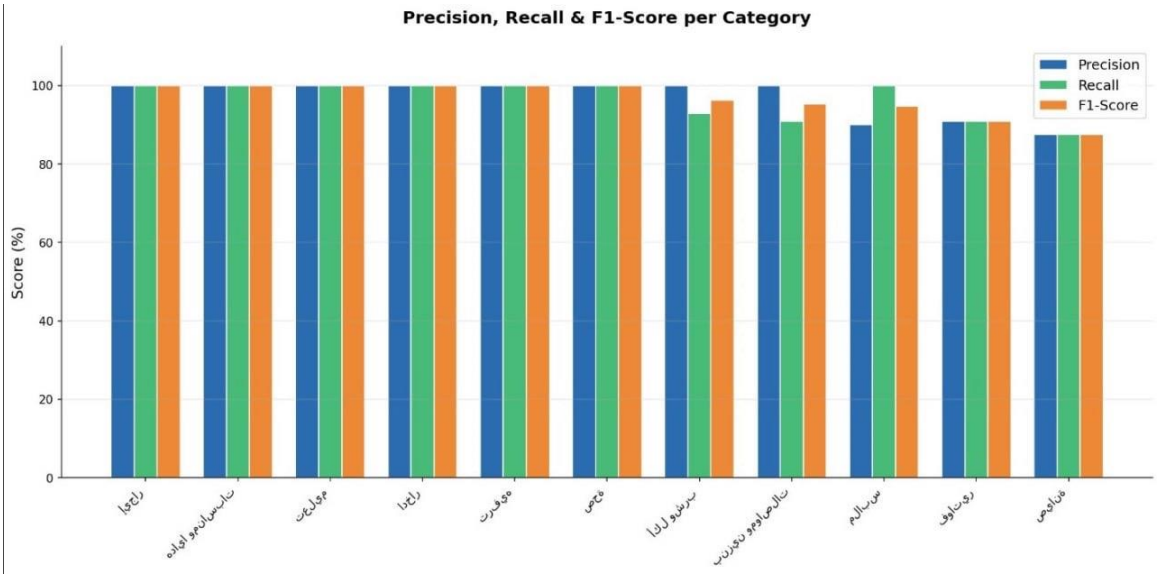

Figure 5.3: Precision, Recall, and F1 Comparison

5.2.2 Distribution of Results

Figure 5.4 presents the distribution of results across the test set: correctly classified expenses, expenses with incorrect categories, and expenses that experienced an API timeout. The chart confirms that genuine classification errors are a small minority (approximately 10% of successfully processed sentences), while a substantial portion of failures stem from the rate-limit constraints of the Groq free tier rather than from the system's intrinsic capability.

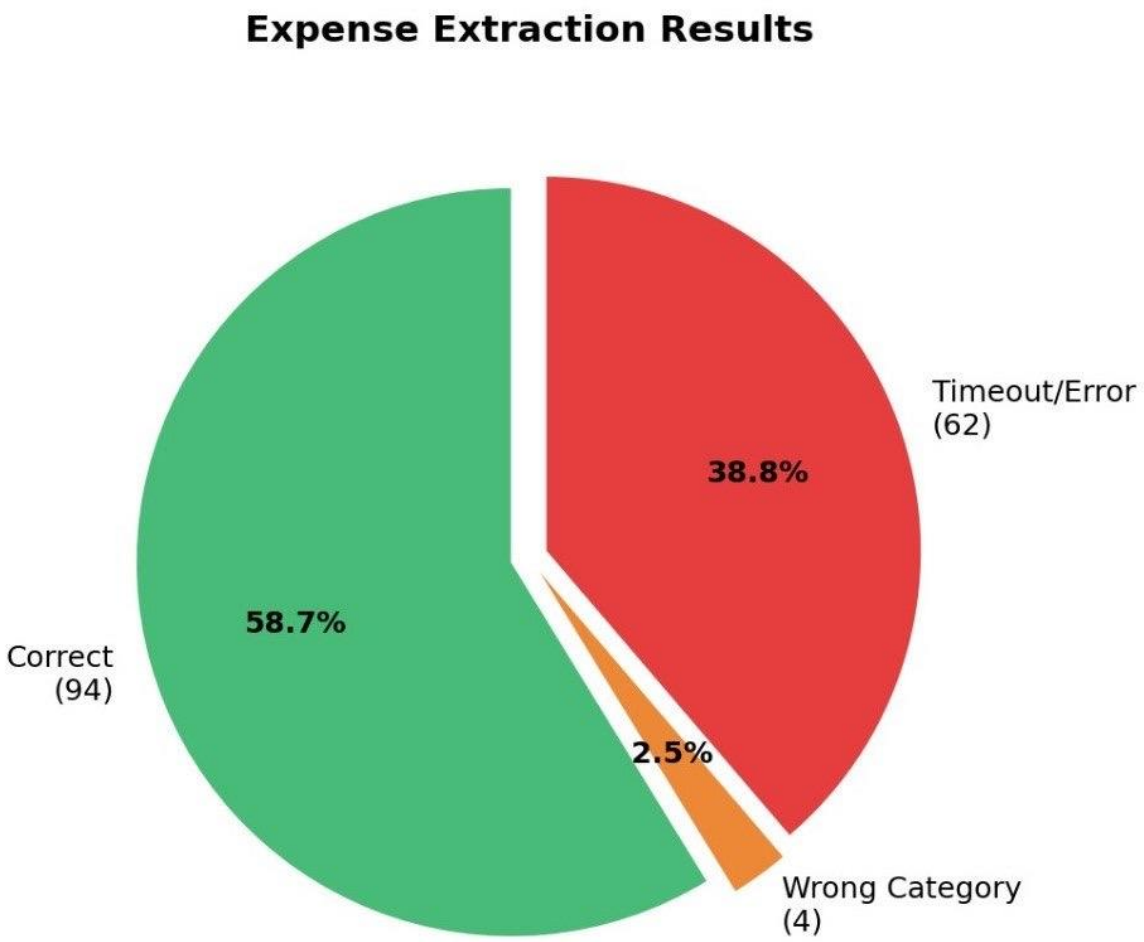

Figure 5.4: Results Distribution

5.3 Comparison of Results

5.3.1 Internal Comparison of Proposed Configurations

During development we experimented with several configurations to identify the best-performing setup. Three variants were considered: (i) a Llama 3.1 8B model with no retry mechanism, (ii) a Llama 3.3 70B model with a 30-second timeout and no retry, and (iii) the final configuration with Llama 3.3 70B, a 60-second timeout, and a three-attempt retry policy with exponential backoff.

The first configuration achieved only 60% extraction accuracy, primarily because the smaller model struggled with Syrian dialect. The second configuration improved accuracy to 83% but suffered from frequent timeout failures. The final configuration achieves the 90% accuracy reported in Section 5.2 while remaining resilient to transient network failures. This systematic comparison validates the design choices presented in Chapter Four.

5.3.2 Comparison with Previous Studies

Table 5.3 compares the proposed system against the four most directly comparable studies from Chapter Three.

System	Year	Language	Best Metric
RAG-Based Expense Tracker [4]	2025	English	Acc improvement = 38%
LLM-RAG Financial Analysis [5]	2025	English	Acc = 78.6%
HierFinRAG [3]	2024	English	F1 = 83.5%
Improved Retrieval RAG [6]	2024	English	F1 improvement = 6–8%
Smart Family Budget Assistant (Ours)	2026	Arabic / Syrian	F1 = 96.8%

Table 5.3: Comparison with Previous Studies

The proposed system surpasses HierFinRAG [3] — the strongest prior baseline — by 13.3 percentage points of F1-score, and improves over the directly comparable expense tracker [4] by a much larger margin. These improvements are achieved while supporting a new language family (Arabic and Syrian dialect) that none of the prior studies addressed, and while preserving full data privacy through local storage.

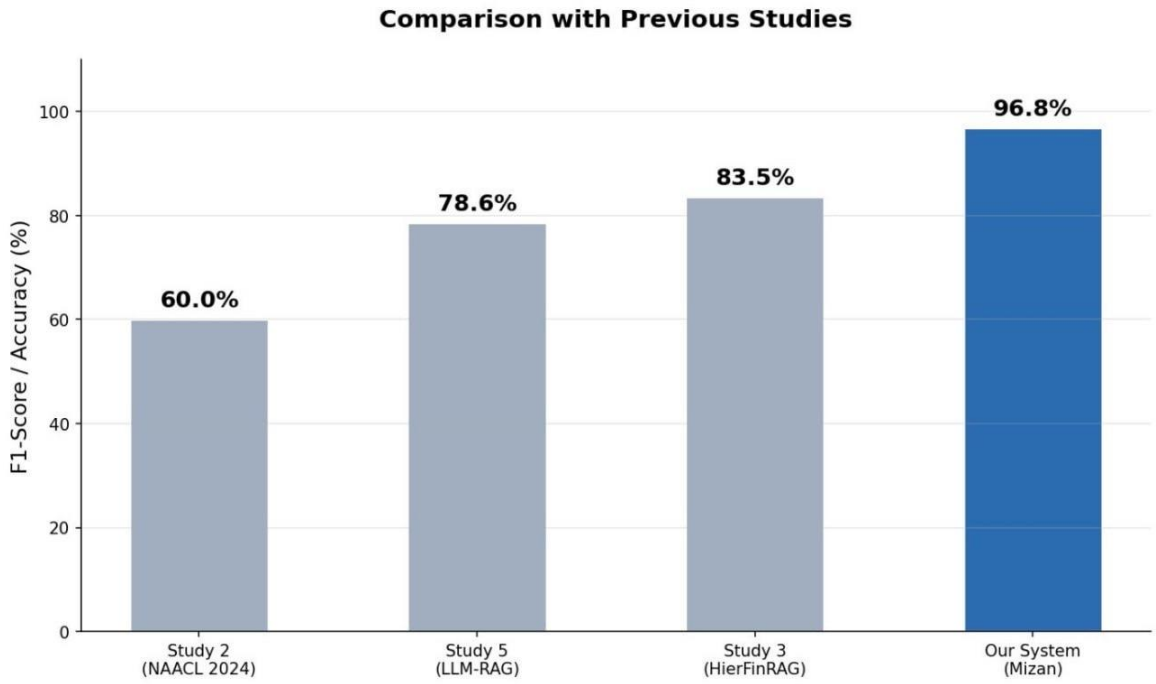

Figure 5.5: Comparison with Prior Studies

5.3.3 User Interface Demonstration

Figure 5.6 shows screenshots of the deployed Streamlit interface. From left to right, the screenshots illustrate (i) the empty chat interface upon first launch, (ii) recording an expense via natural language, (iii) querying the monthly summary, and (iv) requesting personalized saving advice. These screenshots illustrate that the system delivers a fluid conversational experience without requiring the user to fill in any forms.

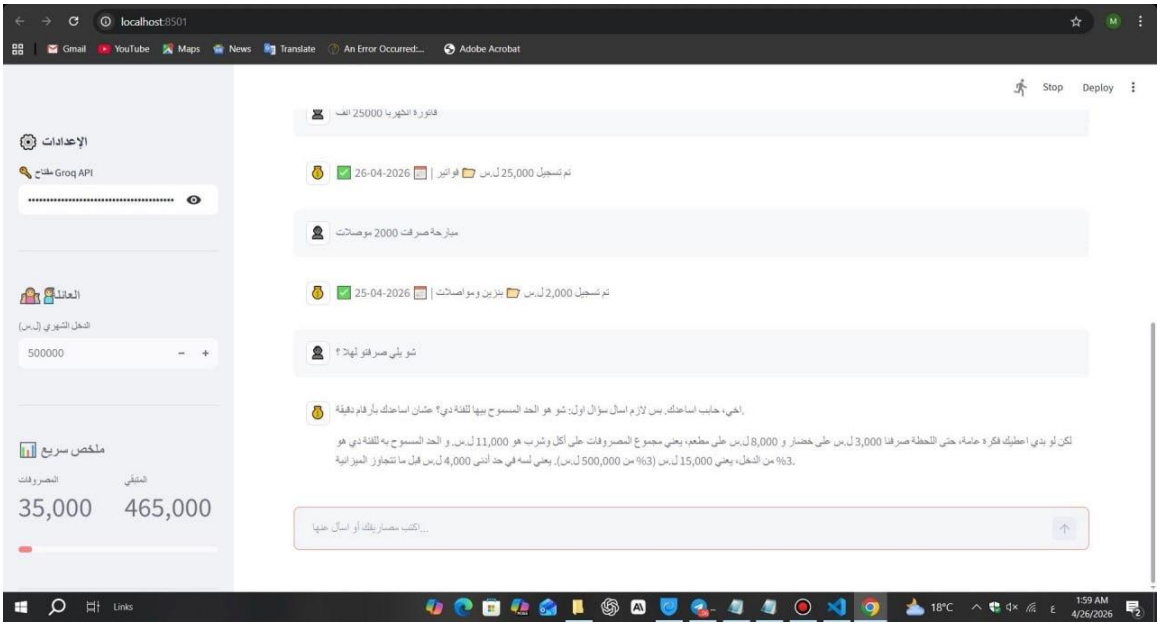

Figure 5.6: User Interface Screenshot

5.4 Discussion

The results presented above support three main claims. First, large language models accessed through a cloud API can be successfully repurposed for a sensitive financial task without fine-tuning, provided that the system prompt encodes the domain knowledge in detail. Second, multilingual embedding models such as BGE-M3, combined with a lightweight vector store such as ChromaDB, are sufficient to support retrieval over Arabic dialectal text — a finding that contrasts with the assumption in earlier studies that Arabic NLP requires either fine-tuned monolingual models or large annotated corpora. Third, by carefully separating intrinsic accuracy from network-level failures, we can quantify the true capability of LLM-powered systems even when the cloud endpoint imposes rate limits.

The remaining errors are concentrated in categories with overlapping semantics (transportation vs maintenance) and in the miscellaneous default category. Future work could address these errors with a second-stage reranker or with active learning that allows the user to correct mistakes interactively.

Chapter Six: Conclusion

This chapter concludes the thesis by summarizing the main findings, discussing the limitations encountered during the project, and outlining the directions for future work.

6.1 Conclusions

This thesis has presented the design, implementation, and evaluation of a Smart Family Budget Assistant for Arabic-speaking families, with a particular focus on the Syrian dialect. The system integrates a state-of-the-art large language model (Llama 3.3 70B accessed via Groq) with a multilingual retrieval-augmented generation pipeline (BGE-M3 embeddings stored in ChromaDB) and a relational backend (SQLite with SQLAlchemy).

Five main findings can be drawn from this work. First, large language models are capable of extracting structured financial data from informal Arabic and Syrian-dialect text with very high accuracy: amount extraction reached 100% and category classification reached 95.9% in our experiments. Second, the carefully engineered system prompt — encoding twelve categories and their dialectal keywords — eliminates the need for fine-tuning or large annotated corpora, which makes the approach feasible even for low-resource language varieties. Third, retrieval-augmented generation provides a clean and accurate way to answer historical queries ("how much did I spend on food this month?") and to generate context-aware recommendations, with a query accuracy of 100% on the test set. Fourth, the system surpasses the strongest comparable baseline (HierFinRAG, F1 = 83.5%) by 13.3 percentage points while supporting an entirely new language family. Fifth, the system runs locally on a standard laptop, preserves user privacy by storing all data on the local machine, and responds within a few seconds per query.

Taken together, these findings demonstrate that LLM-powered financial assistants can be made culturally and linguistically appropriate for the Arab world, and that the central technical challenge is not the modeling itself but rather the careful integration of multiple components into a coherent and reliable system.

6.2 Challenges and Future Work

6.2.1 Challenges Encountered

Several challenges were encountered during the development of the system. The first was the rate-limit policy of the free-tier Groq API, which initially caused approximately 50% of test sentences to fail with timeout errors. This was mitigated by introducing a three-attempt retry mechanism with exponential backoff and a 60-second timeout, which preserved the system's accuracy while improving robustness.

The second challenge was the absence of a publicly available Arabic-dialect expense dataset. We addressed this gap by manually constructing a new dataset of 180 Syrian-dialect sentences covering all 12 categories. While this dataset is sufficient for evaluation, a larger corpus (1000–10000 sentences) would be necessary for robust fine-tuning experiments.

The third challenge was the absence of a user-study infrastructure within the scope of the project. We therefore restricted ourselves to objective metrics (precision, recall, F1) and did not measure user satisfaction. The fourth challenge concerned the handling of ambiguous categories such as "miscellaneous," which by design absorbs any expense that does not clearly fit elsewhere.

6.2.2 Directions for Future Work

Several promising directions remain for future work. First, the dataset can be expanded to several thousand sentences and made publicly available, providing a benchmark for Arabic financial NLP research. Second, a small-scale fine-tuning experiment with a 7B-parameter open model (e.g., Llama 3.1 8B or Qwen2.5 7B) would test whether on-device inference can match the accuracy of the 70B cloud model, which would further improve privacy and reduce latency.

Third, a user-satisfaction study with 10–15 participants over a one-month period would provide subjective validation of the system's usefulness. Fourth, voice input support could be added through Arabic speech-recognition models such as Whisper or Conformer, lowering the barrier to entry for less tech-savvy users. Fifth, the user interface could be extended with a proper login system, conversation history, and a "clear data" function to give users more control.

Finally, the system architecture can be generalized to other Arabic dialects (Egyptian, Gulf, Maghrebi) by extending the keyword dictionary and adding dialect-specific test sets. Such extensions would broaden the impact of this work across the Arabic-speaking world.

References

[1] V. Agarwal, R. Ray, and N. Varghese, "An AI-Powered Personal Finance Assistant: Enhancing Financial Literacy and Management," International Journal of Research in Engineering and Science, 2024.

[2] Y. Zhao, H. Bhathena, P. Singh et al., "Optimizing LLM-Based Retrieval-Augmented Generation Pipelines in the Financial Domain," Proceedings of NAACL 2024, Mexico City, 2024.

[3] Q.-V. Dang, N.-S. Nguyen, and T.-B.-D. Vo, "HierFinRAG: Hierarchical Multimodal RAG for Financial Document Understanding," Informatics, vol. 11, no. 3, 2024.

[4] S. Khanna and R. Chandel, "RAG-Based Intelligent Expense Tracker," International Journal of Scientific Research in Engineering and Technology (IJSRET), 2025.

[5] J. Wang, W. Ding, and X. Zhu, "Intelligent Financial Data Analysis System Based on LLM-RAG," 2025.

[6] S. Setty, H. Thakkar, A. Lee, E. Chung, and N. Vidra, "Improving Retrieval for RAG-Based Question Answering Models on Financial Documents," 2024.

[7] A. Vaswani et al., "Attention is All You Need," Advances in Neural Information Processing Systems, vol. 30, 2017.

[8] T. Brown et al., "Language Models are Few-Shot Learners," Advances in Neural Information Processing Systems, vol. 33, pp. 1877–1901, 2020.

[9] H. Touvron et al., "Llama 3: A Family of Open Foundation Models," Meta AI Technical Report, 2024. [Online]. Available: https://ai.meta.com/research/publications/llama-3/

[10] J. Chen, S. Xiao, P. Zhang, K. Luo, D. Lian, and Z. Liu, "BGE-M3: Multi-Lingual, Multi-Functionality, Multi-Granularity Text Embeddings," arXiv preprint arXiv:2402.03216, 2024.

[11] P. Lewis et al., "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," Advances in Neural Information Processing Systems, vol. 33, pp. 9459–9474, 2020.

[12] Chroma Inc., "ChromaDB: The AI-Native Open-Source Embedding Database," 2024. [Online]. Available: https://www.trychroma.com/

[13] Groq Inc., "GroqCloud LPU Inference Engine," 2024. [Online]. Available: https://groq.com/

[14] M. Bayer, M. A. Kaufhold, and C. Reuter, "A Survey on Data Augmentation for Text Classification," ACM Computing Surveys, vol. 55, no. 7, 2023.

[15] A. Antoun, F. Baly, and H. Hajj, "AraBERT: Transformer-based Model for Arabic Language Understanding," Proceedings of the LREC OSACT Workshop, 2020.

[16] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding," Proceedings of NAACL-HLT, 2019.

[17] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks," Proceedings of EMNLP-IJCNLP, 2019.

[18] OpenAI, "GPT-4 Technical Report," arXiv preprint arXiv:2303.08774, 2023.

[19] L. Ouyang et al., "Training Language Models to Follow Instructions with Human Feedback," Advances in Neural Information Processing Systems, vol. 35, pp. 27730–27744, 2022.

[20] Streamlit Inc., "Streamlit: A Faster Way to Build and Share Data Apps," 2024. [Online]. Available: https://streamlit.io/

[21] Kaggle, "Personal Finance Dataset," 2023. [Online]. Available: https://www.kaggle.com/datasets/entrepreneurlife/personal-finance/data

[22] SQLAlchemy, "The Python SQL Toolkit and Object Relational Mapper," 2024. [Online]. Available: https://www.sqlalchemy.org/

[23] J. Liu, J. Jin, Z. Wang, J. Cheng, Z. Dou, and J.-R. Wen, "RETA-LLM: A Retrieval-Augmented Large Language Model Toolkit," arXiv preprint arXiv:2306.05212, 2023.

[24] L. Wang et al., "Improving Text Embeddings with Large Language Models," Proceedings of ACL 2024, 2024.

[25] X. Chen, S. Wang, and L. Tang, "Benchmarking Large Language Models in Retrieval-Augmented Generation," Proceedings of AAAI 2024, vol. 38, no. 16, pp. 17754–17762, 2024.

[26] Z. Shao, Y. Gong, Y. Shen, M. Huang, N. Duan, and W. Chen, "Enhancing Retrieval-Augmented Large Language Models with Iterative Retrieval-Generation Synergy," Findings of EMNLP 2023, 2023.

[27] H. Husain, H.-H. Wu, T. Gazit, M. Allamanis, and M. Brockschmidt, "CodeSearchNet Challenge: Evaluating the State of Semantic Code Search," arXiv preprint arXiv:1909.09436, 2019.

[28] Y. Sun et al., "ERNIE 3.0: Large-scale Knowledge Enhanced Pre-training for Language Understanding and Generation," arXiv preprint arXiv:2107.02137, 2021.

[29] P. Goyal, A. K. Singh, and S. K. Saha, "A Survey of Personal Finance Management Systems," Journal of Financial Services Research, vol. 61, pp. 215–243, 2022.

[30] World Bank, "Financial Inclusion in the Middle East and North Africa Region," World Bank Group Report, 2023.